다듬(DaDeum) 완전정복

0장. 이 책을 읽는 법과 전체 지도
  0.1 이 책의 약속
  0.2 다듬은 무엇인가
  0.3 전체 지도
  0.4 이 책의 구성
1장. 코드 기초: 이 프로젝트를 읽는 데 필요한 것만
  1.1 프로그램은 무엇인가
  1.2 변수: 값에 이름 붙이기
  1.3 자료형: 문자열, 숫자, 참/거짓
  1.4 조건문: 만약 ~라면
  1.5 반복문: 여러 번 하기
  1.6 목록·집합·사전
  1.7 함수: 이름 붙인 작업 묶음
  1.8 클래스와 dataclass: 묶음 자료
  1.9 모듈과 import
  1.10 정규식: 글자 모양 찾기
  1.11 정렬과 람다
  1.12 예외 처리
  1.13 이 장에서 못 다룬 것 (나오면 그때 설명합니다)
  1.14 확인 문제
2장. 한글이 컴퓨터 안에서 어떻게 생겼나
  2.1 컴퓨터는 글자를 번호로 저장한다
  2.2 11,172는 어디서 나왔나
  2.3 받침 번호를 꺼내는 한 줄
  2.4 실제 코드: `app/hangul.py`

2.5 이걸로 조사를 어떻게 고르나
2.6 율/률도 같은 계산
2.7 숫자는 왜 판단하지 않는가
2.8 어절 경계 찾기
2.9 확인 문제
3장. 형태소 분석: 문장을 조각내고 이름표 붙이기
3.1 왜 필요한가
3.2 용어
3.3 실제 분석 결과
3.4 품사 태그표 (다듬 코드에 나오는 것들)
3.5 같은 글자, 다른 분석: 다듬이 품사를 보는 이유
3.6 Kiwi가 틀린 문장을 어떻게 읽는가
3.7 요청 한 번에 분석은 한 번만: 캐시
3.8 Kiwi 준비 운동
3.9 문장 나누기와 토큰 자르기 도우미 (`app/morphology.py`)
3.10 확인 문제
4장. 서버와 API: 요청 하나가 지나가는 길
4.1 서버와 클라이언트
4.2 API란
4.3 주문서와 음식의 모양: `schemas.py`
4.4 서버 뼈대: `app/main.py`
4.5 레이트리밋: `ratelimit.py`
4.6 `/api/check` 한 줄씩
4.7 긴 글: `analyze_long`
4.8 `/api/health` : 지금 어느 층이 켜져 있나
4.9 직접 따라 하기
4.10 확인 문제
5장. 엔진 뼈대: 후보를 모으고 정리하는 구조
5.1 한 문장으로
5.2 왜 “하나의 흐름”인가: 만들어진 이유
5.3 자료: `RawIssue`
5.4 검사기 등록: `GENERATORS`
5.5 `analyze()` 한 줄씩
5.6 병합: `merge_and_dedupe` (엔진의 심장)

5.7 출처 우선순위표: `sources.py`

5.8 마지막 청소: `_drop_noise_adjacent_to_fixed_typos`

5.9 예문 전체 추적

5.10 확인 문제

0장. 이 책을 읽는 법과 전체 지도

0.1 이 책의 약속

이 책은 **코드를 한 줄도 모르는 사람**이 끝까지 읽으면 다듬(DaDeum)의 코드를 전부 설명할 수 있게 되는 것을 목표로 합니다. 그래서 모든 장이 같은 순서로 진행됩니다.

순서	묻는 것	예
1	**무엇을** 하는가 (목적)	"받침을 보고 을/를을 고른다"
2	**왜** 필요한가 (이유)	"이 규칙이 없으면 `학교을` 을 못 잡는다"
3	**어떻게** 하는가 (로직)	실제 코드와 한 줄씩 풀이
4	**어떻게 확인했는가** (근거)	실제로 돌려 본 결과, 측정 수치
5	**무엇이 문제였고 어떻게 풀었는가** (해결)	실패 사례와 고친 방법

코드 인용은 모두 실제 저장소(`zmflm0110/dadeum`, master 6af0ea5)에서 가져왔습니다. 설명에 나오는 실행 결과는 제가 그 코드를 직접 돌려서 얻은 값입니다. 돌려 보지 않은 것은 "읽은 바로는"이라고 적습니다.

> 이 책에서 가장 중요한 습관 하나: **"그렇다더라"로 넘어가지 않고, 입력을 넣고 출력을 보는 것.** 다듬 프로젝트 자체가 그 습관으로 만들어졌습니다. (13장에서 그 이야기를 합니다.)

0.2 다듬은 무엇인가

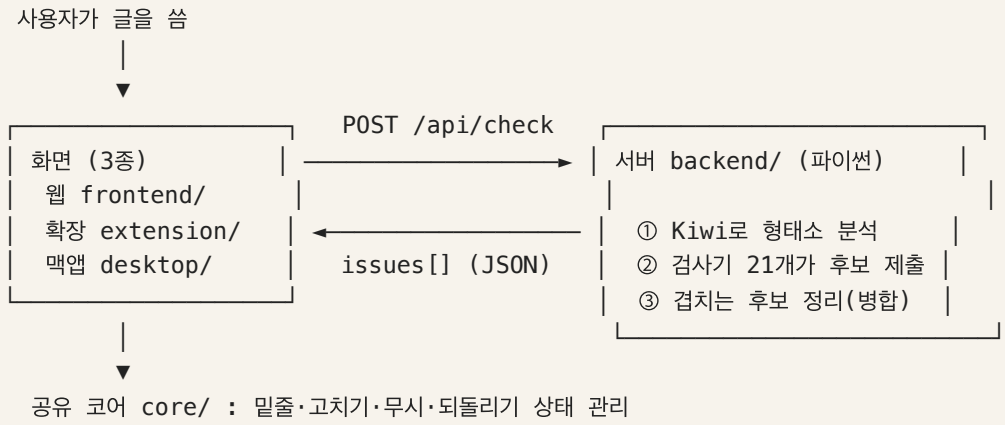
한국어 글을 넣으면 **맞춤법·띄어쓰기·조사·문법 오류에 밑줄을 긋고, 고친 말을 제안하는 프로그램**입니다. 웹 페이지, 크롬 확장, 맥 앱 세 가지 모습이 있고, 셋 다 같은 "엔진"을 씁니다. 전부 내 컴퓨터 안에서 돌고 외부 AI를 부르지 않습니다.

한 문장을 넣으면 이렇게 됩니다.

```
입력:  학교을 가는길에 친구를 만낫는데 그렇게 하면 안되요.

출력:  2-3   '을'     → '를'      (조사, 확실)
       25-26 '안'     → '안 '     (띄어쓰기, 확실)
       26-28 '되요'   → '돼요'    (맞춤법, 확실)
       13-17 '만낫는데' → (없음)   (이상한 말, 애매)
```

0.3 전체 지도



폴더	한 줄 설명	이 책의 장
`backend/app/`	교정 엔진 본체	4-10장
`backend/ml/`	모델을 학습시키는 코드	7-9장
`backend/eval/`	엔진이 좋은지 재는 도구	11장
`backend/tests/`	자동 테스트 997개	11장
`core/`	화면 세 개가 공유하는 상태기계	12장
`frontend/` `extension/` `desktop/`	화면	12장

0.4 이 책의 구성

장	내용
1	코드 기초: 이 프로젝트를 읽는 데 필요한 파이썬 문법만
2	한글이 컴퓨터 안에서 어떻게 생겼나 (받침 계산의 원리)
3	형태소 분석: 문장을 조각내고 이름표를 붙이기
4	서버와 API: 요청 하나가 지나가는 길
5	엔진 뼈대: 후보를 모으고 정리하는 구조
6	규칙 검사기: 조사·자모·사전·의존명사 등
7	머신러닝 기초와 혼동쌍 모델
8	단어 경계 모델
9	오타·사전·언어모델·이상탐지
10	띄어쓰기
11	평가: 좋은지 어떻게 아는가
12	화면과 공유 코어
13	개발 과정과 교훈
14	용어집, 직접 돌려 보기, 연습문제

앞 장을 건너뛰면 뒤 장의 용어를 못 알아볼 수 있습니다. 처음엔 순서대로 읽으세요.

1장. 코드 기초: 이 프로젝트를 읽는 데 필요한 것만

다듬의 엔진은 **파이썬**으로 쓰여 있습니다. 파이썬 전체를 배울 필요는 없고, 이 프로젝트에 실제로 나오는 문법만 알면 됩니다. 각 항목마다 "다듬 코드의 어디에 나오는지"를 같이 적었습니다.

1.1 프로그램은 무엇인가

프로그램은 **입력을 받아 정해진 순서대로 처리해서 출력을 내는 글**입니다. 요리 레시피와 같습니다.

```
입력: "학교을 갔다"  →  [처리: 받침을 보고 을/를 고르기]  →  출력: "학교를 갔다"
```

컴퓨터는 사람처럼 "대충" 이해하지 못합니다. 모든 단계를 정확히 적어 줘야 합니다. 다듬 코드가 길어 보이는 이유는 "이 경우는 이렇게, 저 경우는 저렇게"를 전부 적었기 때문입니다.

1.2 변수: 값에 이름 붙이기

```
text = "학교을 갔다"
count = 3
```

`text`라는 이름표를 값에 붙인 것입니다. `=`는 "같다"가 아니라 **"오른쪽 값을 왼쪽 이름에 넣어라"**입니다.

다듬 예: `app/checkers/confusion.py`에는 `_DOE_FORCES_DOE = set("고게기니나냐네는면며자거든어었겠시")`가 있습니다. "되 뒤에 이 글자들이 오면 무조건 '되'"라는 목록에 이름을 붙인 것입니다.

1.3 자료형: 문자열, 숫자, 참/거짓

종류	예	다듬에서
문자열 `str`	`"학교"`	입력 글, 제안, 설명 메시지
정수 `int`	`26`	글자 위치(start, end)
실수 `float`	`0.97`	확률, 문턱
참/거짓 `bool`	`True`	조건 검사 결과
없음 `None`	`None`	“답이 없다” (예: 받침을 알 수 없는 글자)

문자열의 번호(인덱스)는 0부터 셉니다.

```
text = "학교을 갔다"
text[0]      # '학'
text[2]      # '을'
text[0:2]    # '학교'   (0번부터 2번 "직전"까지)
```

다듬이 말하는 `start=2, end=3`은 “2번 글자부터 3번 직전까지”, 즉 `text[2:3]` = `'을'` 입니다. 이 “끝은 포함하지 않는다” 규칙이 프로젝트 전체에서 일관되게 쓰입니다.

1.4 조건문: 만약 ~라면

```
if idx == NO_BATCHIM_INDEX:
    answer = "를"
else:
    answer = "을"
```

`==`는 “같은가?”를 묻는 기호입니다(`=`와 다릅니다). 다듬 코드 거의 모든 줄이 이런 조건의 연속입니다.

`and`(둘 다), `or`(하나라도), `not`(반대)로 조건을 묶습니다.

```
if actual in (no_batchim, has_batchim) and actual != expected:
    ...   # 조사가 둘 중 하나이면서, 기대한 것과 다르면
```

1.5 반복문: 여러 번 하기

```
for ch in "학교을":
    print(ch)
# 학
# 교
# 을
```

`for 이름 in 목록:`은 목록의 원소를 하나씩 꺼내 아래 들여쓴 줄을 실행합니다. **들여쓰기(맨 앞 공백)가 문법입니다.** 파이썬은 괄호 대신 들여쓰기로 "여기까지가 for의 안쪽"을 표시합니다.

`continue`는 "이번 차례는 건너뛰고 다음으로", `break`는 "반복을 완전히 끝내기"입니다. 다음의 검사기들은 `if (오탐 위험 조건): continue`로 위험한 자리를 건너뛰는 모양이 아주 많습니다.

1.6 목록·집합·사전

```
issues = [ ... ]            # 목록(list): 순서 있음, 같은 값 여러 개 가능
seen = {3, 7, 9}            # 집합(set): 순서 없음, 중복 없음, "있나?" 확인이 빠름
rule = {"pattern": "결재", "replacement": "결제"}   # 사전(dict): 이름표 → 값
```

	만들기	꺼내기	다듬 예
list	`[1, 2]`	`a[0]`	검사기가 낸 `issues`
set	`{1, 2}`	`3 in a`	`seen_spans` : 이미 본 위치
dict	`{"k": v}`	`d["k"]`	`dict_rules.json`의 규칙 한 개

리스트 컴프리헨션은 "목록을 걸러서 새 목록 만들기"를 한 줄로 쓴 것입니다.

```
actionable = [i for i in issues if i.confidence != "low"]
# 풀어 쓰면:
actionable = []
for i in issues:
    if i.confidence != "low":
        actionable.append(i)
```

1.7 함수: 이름 붙인 작업 묶음

```python
def final_index(ch):
    """받침 번호를 돌려준다."""
    if not is_hangul_syllable(ch):
        return None
    return (ord(ch) - 0xAC00) % 28
```

- `def 이름(입력):`으로 만듭니다. `return`은 결과를 돌려주고 함수를 끝냅니다.
- 따옴표 세 개 `"""..."""`는 **설명문(독스트링)**입니다. 실행에 영향이 없고 읽는 사람을 위한 것입니다. 다듬 코드는 이 설명이 매우 깁니다. 왜 이렇게 했는지, 어떤 실패가 있었는지가 거기 적혀 있습니다.
- `#` 뒤는 한 줄 주석입니다.

함수를 쓰는 것을 **호출**이라고 합니다: `final_index("교")` → `0`.

타입 힌트

```python
def check_josa(text: str, kiwi: Kiwi) -> list[RawIssue]:
```

`: str`은 "이 입력은 문자열", `-> list[RawIssue]`는 "결과는 RawIssue의 목록"이라는 **표시**입니다. 실행에 영향은 없고 읽기 쉽게 하려는 것입니다. `str | None`은 "문자열이거나 None"입니다.

1.8 클래스와 dataclass: 묶음 자료

관련된 값 여러 개를 하나로 묶고 싶을 때 씁니다. 다듬의 핵심 자료인 `RawIssue`가 이렇게 생겼습니다(`app/checkers/base.py`).

```python
from dataclasses import dataclass

@dataclass
class RawIssue:
    start: int
    end: int
    category: str
    source: str
    original: str
    suggestions: list[str]
    message: str
    confidence: str
```

"지적 한 건"은 위치(start, end), 종류(category), 출처(source), 원문(original), 고칠 말(suggestions), 설명(message), 확신도(confidence)로 이루어집니다.

```
r = RawIssue(start=2, end=3, category="josa", source="josa_rule",
             original="을", suggestions=["를"], message="...", confidence="h
         igh")
r.original      # '을'  ← 점(.)으로 안의 값을 꺼냄
r.suggestions[0]  # '를'
```

`@dataclass`는 "이 클래스의 생성자와 출력 모양은 자동으로 만들어 줘"라는 표시입니다. `@dataclass(frozen=True)`는 "한 번 만들면 값을 못 바꾼다"입니다(`Generator`, `Source`가 이렇습니다).

1.9 모듈과 import

파일 하나가 모듈입니다. 다른 파일의 것을 가져다 쓰려면 `import`합니다.

```
from .base import RawIssue          # 같은 폴더의 base.py에서 RawIssue를
from ..hangul import final_index    # 한 단계 위 폴더의 hangul.py에서
from kiwipiepy import Kiwi          # 설치된 외부 라이브러리에서
```

점(`.`)의 개수는 폴더 거리입니다. 한 개는 같은 폴더, 두 개는 위 폴더입니다.

1.10 정규식: 글자 모양 찾기

정규식(regular expression)은 "이런 모양의 글자를 찾아라"를 적는 작은 언어입니다. 다듬의 사전 규칙 474개가 전부 정규식입니다.

기호	뜻	예
`abc`	그 글자 그대로	`결재`
`A|B`	A 또는 B	`결재|결제`
`[가-힣]`	한글 한 글자	
`.`	아무 글자	
`+`	앞의 것 1번 이상	`[가-힣]+`
`^` `$`	문자열의 처음, 끝	
`(?<=X)`	"바로 앞에 X가 있을 때만" (앞을 확인만 하고 소비하지 않음)	
`(?=X)`	"바로 뒤에 X가 있을 때만"	
`(?!X)`	"바로 뒤에 X가 없을 때만"	

다듬의 실제 규칙(`data/dict_rules.json`)입니다.

```
(?<=카드로 )결재        → 결제
```

"`카드로` 바로 뒤에 오는 `결재`"만 찾아서 `결제`로 바꿉니다. 앞의 `카드로`는 확인만 하고 바꾸는 범위에는 들어가지 않습니다. 그래서 `서류를 결재`는 건드리지 않습니다. **문맥을 정규식 안에 넣어서** 같은 철자의 다른 쓰임을 구별하는 것입니다.

파이썬에서는 이렇게 씁니다.

```python
import re
pat = re.compile(r"(?<=카드로 )결재")
for m in pat.finditer("카드로 결재했다"):
    print(m.start(), m.end(), m.group(0))   # 4 6 결재
```

1.11 정렬과 람다

```python
ordered = sorted(issues, key=lambda i: (i.start, -(i.end - i.start)))
```

`sorted`는 목록을 정렬합니다. `key=`는 "무엇을 기준으로 줄 세울지"입니다. `lambda i: ...`는 이름 없는 한 줄짜리 함수입니다. 위 기준은 `(시작 위치, 길이의 음수)`이므로 **시작이 빠른 것이 먼저, 시작이 같으면 긴 것이 먼저** 옵니다. 5장의 병합이 이 정렬 위에서 돌아갑

니다.

1.12 예외 처리

```
try:
    return kiwi.analyze(word)[0][1]
except Exception:
    return 0.0
```

`try` 안에서 오류가 나도 프로그램이 죽지 않고 `except` 안쪽을 실행합니다. 다음은 "분석 한 번이 실패해도 서버 전체가 죽으면 안 된다"는 원칙으로 곳곳에 씁니다.

1.13 이 장에서 못 다룬 것 (나오면 그때 설명합니다)

- `yield` (값을 하나씩 내보내는 함수): 8장 경계 후보
- 데코레이터 `@router.post(...)` : 4장 서버
- `async` / `await` (JavaScript): 12장 화면

1.14 확인 문제

1. `text = "안녕하세요"` 일 때 `text[1:3]` 은? (답: `녕하`)
2. `start=4, end=6` 인 지적이 가리키는 글자를 `"카드로 결재했다"` 에서 꺼내면? (답: `결재`)
3. `[i for i in [1,2,3,4] if i > 2]` 의 결과는? (답: `[3, 4]`)

2장. 한글이 컴퓨터 안에서 어떻게 생겼나

다듬의 조사 검사(`을/를`, `이/가`…)는 한 가지 계산에 기대고 있습니다. **"이 글자에 받침이 있는가?"를 산수로 알아내는 것**입니다. 이 장은 그 산수를 처음부터 설명합니다.

2.1 컴퓨터는 글자를 번호로 저장한다

컴퓨터 안에는 글자가 없고 **번호**만 있습니다. 글자마다 번호를 하나씩 정해 둔 표를 **유니코드**라고 합니다. 파이썬에서 `ord("가")`는 글자의 번호를, `chr(번호)`는 번호의 글자를 돌려줍니다.

```
ord("가")   # 44032  (16진수로 0xAC00)
ord("힣")   # 55203  (16진수로 0xD7A3)
chr(44032)  # '가'
```

완성된 한글 글자(가, 각, 간, …, 힣)는 **44032번부터 55203번까지 11,172개**가 빈틈없이 줄지어 있습니다.

2.2 11,172는 어디서 나왔나

한글 한 글자는 세 부분으로 이루어집니다.

부분	이름	가짓수	예 (학)
첫소리	초성	19	ㅎ
가운뎃소리	중성	21	ㅏ
끝소리(받침)	종성	28 (받침 없음 포함)	ㄱ

19 × 21 × 28 = **11,172**. 모든 조합이 번호표에 하나씩 들어 있는 것입니다.

번호는 이 순서로 매겨져 있습니다.

```
(글자 번호 − 44032) = (초성 번호 × 21 + 중성 번호) × 28 + 종성 번호
```

사전에서 "초성 → 중성 → 종성" 순으로 정렬한 것과 같습니다. 같은 초성, 같은 중성인 글자들이 종성 순서대로 28개씩 붙어 있습니다.

```
가(종성 0, 받침 없음) 각(1: ㄱ) 갂(2: ㄲ) 갃(3: ㄳ) 간(4: ㄴ) ... 갛(27: ㅎ)
```

2.3 받침 번호를 꺼내는 한 줄

위 식에서 **28로 나눈 나머지가 종성 번호**입니다.

```
받침번호 = (ord(글자) − 0xAC00) % 28
```

`%`는 나머지입니다. 실제로 계산해 보았습니다.

글자	번호	번호 − 44032	% 28	뜻
가	44032	0	**0**	받침 없음
각	44033	1	**1**	ㄱ
힣	55203	11171	**27**	ㅎ
교	44368	336	**0**	받침 없음
학	54617	…	**1**	ㄱ
을	51012	…	**8**	ㄹ

종성 번호표는 이렇습니다(0은 받침 없음).

```
0:없음 1:ㄱ 2:ㄲ 3:ㄳ 4:ㄴ 5:ㄵ 6:ㄶ 7:ㄷ 8:ㄹ 9:ㄺ 10:ㄻ 11:ㄼ 12:ㄽ 13:ㄾ 14:ㄿ
15:ㅀ 16:ㅁ 17:ㅂ 18:ㅄ 19:ㅅ 20:ㅆ 21:ㅇ 22:ㅈ 23:ㅊ 24:ㅋ 25:ㅌ 26:ㅍ 27:ㅎ
```

2.4 실제 코드: `app/hangul.py`

이 파일 전체가 38줄입니다. 핵심만 옮깁니다.

```
HANGUL_BASE = 0xAC00      # '가'
HANGUL_LAST = 0xD7A3      # '힣'
JONGSEONG_COUNT = 28      # 받침 가짓수 (0 = 받침 없음)

NO_BATCHIM_INDEX = 0
NIEUN_INDEX = 4           # ㄴ – 을/를 예외
RIEUL_INDEX = 8           # ㄹ – 로/으로 예외

def is_hangul_syllable(ch):
    return len(ch) == 1 and HANGUL_BASE <= ord(ch) <= HANGUL_LAST

def final_index(ch):
    if not is_hangul_syllable(ch):
        return None
    return (ord(ch) - HANGUL_BASE) % JONGSEONG_COUNT
```

읽는 법:

1. `is_hangul_syllable` : “글자 하나이면서 번호가 44032~55203 사이인가?” 한글 완성 글자인지 확인합니다.
2. `final_index` : 한글이 아니면 `None` (모른다)을 돌려주고, 한글이면 받침 번호를 돌려줍니다.

왜 한글이 아닐 때 `None` 인가? 숫자·영문·기호는 받침이 있는지 알 수 없기 때문입니다. 이 엔진의 원칙은 “모르면 가만히 있기”이고, `None` 이 그 신호입니다. 부르는 쪽은 `None` 을 받으면 판단을 포기합니다.

2.5 이걸로 조사를 어떻게 고르나

조사는 앞말의 마지막 글자 받침에 따라 짝이 정해집니다.

받침 없음(0)	받침 있음
를	을
는	은
가	이
와	과
로	으로

`학교을` 의 마지막 글자는 `교` , 받침 번호는 0, 따라서 `를` **이어야 합니다**. 그런데 `을` 이 적혀 있으니 오류입니다.

`로/으로`만 예외가 하나 있습니다. **ㄹ 받침(8번)은 받침 없는 것처럼 `로`를 씁니다.** `물로`, `서울로`가 그렇습니다. 코드에서는 `rieul_exception=True`일 때 `idx == RIEUL_INDEX`도 "받침 없음"으로 취급합니다.

```
def _expected_particle(host_last_char, no_batchim, has_batchim, rieul_exce
        ption=False):
    idx = _final_index(host_last_char)
    if idx is None:
        return None           # 숫자/영문/기호: 모른다
    no_batchim_case = idx == NO_BATCHIM_INDEX or (rieul_exception and idx
        == RIEUL_INDEX)
    return no_batchim if no_batchim_case else has_batchim
```

2.6 율/률도 같은 계산

`합격률`, `비율`: 모음이나 ㄴ 받침 뒤에는 `율`, 그 밖에는 `률`을 씁니다(맞춤법 제11항). `NIEUN_INDEX = 4`가 이 규칙의 예외를 위한 상수입니다. 같은 `final_index` 하나로 두 규칙을 처리합니다.

2.7 숫자는 왜 판단하지 않는가

`3을`일까 `3를`일까? 3을 "삼"(ㅁ 받침)으로 읽으면 `을`, "쓰리"로 읽으면 `를`입니다. 코드에는 숫자를 읽는 소리로 푸는 표(`_DIGIT_FINAL`)가 만들어져 있는데 **실제로 쓰지 않습니다.**

```
_DIGIT_FINAL = {"0": 21, "1": 8, "2": 0, "3": 16, ...}  # 영/일/이/삼/…
```

한 번 켜 보고 실측했더니 재현율은 0.850 → 0.866으로 올랐지만 **정문 오탐이 0.05% → 0.15%로 3배**가 됐습니다. 원인은 읽는 법이 하나가 아니라는 것입니다.

문장	읽는 법	현재 표기가 맞는가
언리얼 엔진 3**를**	"쓰리"	맞음 (오류 아님)
충정로 3**가**	"가"(街)는 조사가 아니라 지명 단위	오류 아님
여우원숭이 5**과**	"과"(科)는 분류 단위	오류 아님

그래서 결정은 "문맥 신호가 생기기 전에는 숫자 뒤 조사를 판단하지 않는다"였고, 읽기 유틸은 나중을 위해 남겨 두었습니다. **여기서 배울 것:** 맞출 수 있는 걸 일부러 안 맞춘 것입니다. 이 프로젝트에서 "침묵"은 실패가 아니라 선택입니다.

2.8 어절 경계 찾기

`hangul.py`에는 하나가 더 있습니다.

```
def eojeol_bounds(text, i):
    """i번 글자가 속한, 공백으로 나뉜 한 덩어리의 [시작, 끝)."""
    start = i
    while start > 0 and not text[start - 1].isspace():
        start -= 1
    end = i
    while end < len(text) and not text[end].isspace():
        end += 1
    return start, end
```

`i`에서 왼쪽으로 공백이 나올 때까지, 오른쪽으로 공백이 나올 때까지 걸어갑니다. `while`은 "조건이 참인 동안 반복"입니다.

```
text = "나는 학교를 갔다",  i = 5 (교)
왼쪽으로: 4(학) → 3(공백) 멈춤 → start = 4
오른쪽으로: 5 → 6(를) → 7(공백) 멈춤 → end = 7
결과 (4, 7) = "학교를"
```

2.9 확인 문제

1. `"뽀"`의 받침 번호는? (초성 ㅃ, 중성 ㅗ, 종성 없음 → 0)
2. `"밥을"`에서 `을`이 맞는가? (`밥`의 받침 ㅂ=17, 있음 → `을` 맞음)
3. `"서울로"`가 맞는 이유는? (`울`의 받침 ㄹ=8 → 예외로 `로`)

3장. 형태소 분석: 문장을 조각내고 이름표 붙이기

3.1 왜 필요한가

`학교를`과 `학교에서`와 `학교는`은 글자로는 다른 단어처럼 보이지만 모두 `학교` + 조사입니다. 교정기가 "조사 `을`이 맞는가"를 보려면 먼저 **어디까지가 명사이고 어디부터가 조사인지** 알아야 합니다. 이것을 해 주는 프로그램이 **형태소 분석기**이고, 다듬이 쓰는 것은 **Kiwi**(kiwipiepy 0.23.2, LGPL-3.0)입니다.

> 중요: Kiwi는 다듬이 만든 것이 아닙니다. 다듬의 규칙 검사기 대부분이 Kiwi의 분석 결과 위에서 돌아갑니다. Kiwi가 틀리게 읽으면 그 검사기도 틀립니다. 이 장 뒤쪽이 "Kiwi를 믿을 때와 못 믿을 때"입니다.

3.2 용어

용어	뜻	예
어절	공백으로 나뉜 덩어리	`학교를`
형태소	뜻이나 문법 기능이 있는 가장 작은 조각	`학교` / `를`
품사 태그	형태소의 종류 이름표	`NNG`, `JKO`
어간	활용하는 말의 변하지 않는 부분	`먹`(먹다)
어미	어간 뒤에 붙어 문법 기능을 하는 조각	`었`, `다`, `던`
토큰	분석기가 돌려주는 조각 한 개 (형태소 하나에 해당)	

3.3 실제 분석 결과

`kiwi.tokenize(text)`는 토큰의 목록을 돌려줍니다. 토큰 하나에는 `form`(글자), `tag`(품사), `start`(시작 위치), `len`(길이)이 있습니다. 직접 돌려 본 결과입니다.

```
학교를 갔다
  학교 /NNG  start=0 len=2
  를   /JKO  start=2 len=1
  가   /VV   start=4 len=1
  었   /EP   start=4 len=1
  다   /EF   start=5 len=1
```

- `학교/NNG` : 일반명사. `를/JKO` : 목적격 조사.
- `갔다`가 `가/VV` + `었/EP` + `다/EF`로 쪼개졌습니다. `가다`의 `가`(어간) + 과거 `-았/었-`(선어말어미) + 종결어미 `다`.
- 주목: **`가`와 `었`의 start가 둘 다 4**입니다. `갔`이라는 한 글자 안에 두 형태소가 녹아 있기 때문입니다. 또 과거 어미의 원래 모양은 `았`인데 Kiwi는 `었`으로 정규화해 돌려줍니다. 그래서 **`form`이 원문 글자와 항상 같지는 않습니다.** 원문과 대조해야 할 때 코드가 `text[pos:pos+1]`로 다시 확인하는 이유입니다(`confusion.py`의 `든` 잠금).

3.4 품사 태그표 (다듬 코드에 나오는 것들)

태그	이름	예
NNG	일반명사	학교, 친구
NNP	고유명사	손병호, 서울
NNB	의존명사	것, 수
NP	대명사	나, 이것
NR	수사	하나
VV	동사	먹-, 가-
VA	형용사	예쁘-, 많-
VX	보조용언	있-(먹고 있다)
VCP / VCN	이다 / 아니다	
MAG	일반부사	너무, 매우
MM	관형사	이런, 모든
JKS / JKO / JKB / JKG	주격 / 목적격 / 부사격 / 관형격 조사	가, 를, 에, 의
JX / JC	보조사 / 접속조사	은/는, 와/과
EP	선어말어미	었, 시
EF	종결어미	다, 요
EC	연결어미	고, 는데, 든
ETM	관형사형 전성어미	는, 던, ㄴ
ETN	명사형 전성어미	음, 기
XSN / XSV / XSA	명사 / 동사 / 형용사 파생 접미사	-적, -하, -스럽
SF	문장 끝 부호	. ? !
SN / SL / SH	숫자 / 외국어 / 한자	

`-I`, `-R` 이 붙은 태그(`VA-I`, `VV-R`)는 불규칙 활용 표시입니다. 다듬은 `pos_base(tag)` 로 `-` 뒤를 떼어 씁니다.

```python
def pos_base(tag):
    return tag.split("-")[0]     # 'VV-R' -> 'VV'
```

3.5 같은 글자, 다른 분석: 다듬이 품사를 보는 이유

글자만 봐서는 구분할 수 없고 품사를 봐야 답이 갈리는 경우가 이 프로젝트에 많습니다. 같은 `든`인데 Kiwi가 읽는 방식에 따라 이렇게 갈립니다.

```
봤든 사람   → 보/VV  었/EP  든/ETM  사람/NNG      든이 '관형사형 어미'(ETM)
먹었던 밥   → 먹/VV  었/EP  던/ETM  밥/NNG
```

`든`을 ETM(명사를 꾸미는 어미)으로 읽은 자리는 문법상 `던`이어야 합니다. `든`은 연결어미(EC)라서 명사를 꾸미지 못하기 때문입니다. 이 한 줄이 `confusion.py`의 `_adnominal_den_spans`입니다. "확률이 아니라 문법으로 확정"하는 예입니다.

3.6 Kiwi가 틀린 문장을 어떻게 읽는가

오류가 들어간 문장을 Kiwi는 "모르는 말"로 처리하거나 엉뚱하게 읽습니다. 실제로 이렇습니다.

```
학교을 갔다   → 학교/NNG  을/NNG  가/VV ...
```

`을`이 조사(JKO)가 아니라 명사(NNG)로 읽혔습니다. 틀린 조사를 조사로 인정하지 않은 것입니다. 그래서 조사 검사기는 "조사로 태그된 것을 찾는" 방식으로 만들 수 없습니다. 오류가 있는 바로 그 자리가 조사로 태그되지 않으니까요.

> `josa.py` 주석의 뜻(의역): 틀린 조사는 Kiwi가 다른 것으로 다시 태깅하기 때문에(`학교을`의 `을`이 NNG가 된다), 태그만 훑으면 실제 오류를 전부 놓친다.

그래서 방식을 뒤집습니다. **명사 토큰이 끝나는 바로 다음 글자**를 직접 보고, 그 글자가 조사 후보(`을/를/은/는/이/가/와/과/로/으로`)이면서 받침 규칙과 어긋나는지 봅니다. 6장에서 코드로 봅니다.

3.7 요청 한 번에 분석은 한 번만: 캐시

검사기가 21개인데 각자 `kiwi.tokenize(text)`를 부르면 같은 문장을 21번 분석합니다. 느립니다. 두 장치가 있습니다.

① **`tokenize_cached`** (**`app/kiwi_singleton.py`**)

```
@lru_cache(maxsize=8)
def tokenize_cached(analyzer, text):
    return analyzer.tokenize(text)
```

`@lru_cache`는 "같은 입력이 오면 계산 없이 저번 결과를 돌려주라"는 표시입니다(최근 8개 기억). 주의 문구가 달려 있습니다: **"반환된 목록은 공유되므로 읽기 전용으로 쓸 것."** 한 검사기가 목록을 고치면 다른 검사기가 오염됩니다.

② **`_SharedAnalysis`** (**`app/engine.py`**)

요청 하나 동안만 `tokenize` 결과를 나눠 쓰는 얇은 포장입니다. 검사기들은 `kiwi.tokenize(text)`를 평소처럼 부르는데, 포장이 중간에서 캐시를 끼워 줍니다. 요청마다 새로 만들어서 요청 사이에 메모리가 쌓이지 않습니다.

3.8 Kiwi 준비 운동

```
kiwi = Kiwi(load_default_dict=True, load_typo_dict=False, load_multi_dict=
        False)
kiwi.tokenize("다듬")       # 서버가 뜰 때 한 번 돌려 둔다
```

- Kiwi의 사전을 어디까지 적재할지는 **메모리와 정확도의 교환**입니다. 파일 안 주석에 실측표가 있습니다.

설정	메모리	부팅	FPR(오탐률)
사전 전부	578MB	0.94초	.0009
다중어 사전 끔 (현재)	429MB	0.46초	.0009
기본 사전도 끔	344MB	0.33초	.0014

다중어 사전을 끄면 정밀도·오탐률이 소수 넷째 자리까지 같고 메모리가 149MB 줄어서 껐습니다. 기본 사전은 켜 둡니다. 껐을 때 오탐률이 .0009 → .0013으로 오르는데, 이 엔진이 가장 신경 쓰는 게 오탐이라 "조용히 맞바꾸지 않는다"는 판단입니다.

- 마지막 줄 `kiwi.tokenize("다듬")`은 **미리 한 번 돌려서** 내부 구조를 만들어 두는 것입니다. 안 하면 첫 사용자가 120MB 할당과 약 300ms 지연을 떠안습니다.

3.9 문장 나누기와 토큰 자르기 도우미 (`app/morphology.py`)

```
def sentence_slices(tokens):
    """SF(문장 끝 부호) 뒤마다 끊어 (시작 토큰 번호, 끝 토큰 번호) 목록을 만든다."""
```

문서 전체를 한꺼번에 받으므로 "문장의 마지막 어미"를 보는 검사는 문장 경계를 알아야 합니다. 이 함수가 토큰 목록을 문장 단위로 자릅니다.

```
def tokens_within(tokens, start, end):
    """[start, end) 안에 들어 있는 토큰만."""
```

어절 하나만 따로 분석하지 않고 **문장 분석에서 잘라 쓰는** 이유가 주석에 있습니다: 어절만 따로 분석하면 태그가 달라질 수 있기 때문입니다(`계시`는 문장 안에서 VV, 혼자일 때 VX). 교정이 조용히 안 먹히는 일이 생깁니다.

```
def join_changed(kiwi, morphemes, original):
    """고친 형태소 목록을 kiwi.join으로 글자로 되돌린다. 바뀐 게 없으면 None."""
```

이 세 개가 형태소 기반 교정(`morph_rules.py` 등 6개 검사기)의 공통 부품입니다. "어간을 고친 뒤 `kiwi.join`으로 활용형을 다시 만든다"는 방식이라, 문자열 규칙이 `잊혀지`만 알던 자리의 **모든 활용형**(잊혀졌다, 잊혀지는, 잊혀질…)을 한 규칙으로 잡습니다.

3.10 확인 문제

1. `학교를 갔다`에서 `를`의 품사는? (JKO, 목적격 조사)
2. 왜 `갔다`에서 `가`와 `었`이 같은 start를 가지나? (한 글자에 두 형태소가 녹아 있어서)
3. 틀린 조사를 찾을 때 태그가 JKO인 토큰을 찾으면 안 되는 이유는? (틀린 조사는 NNG 등으로 재해석되어 JKO로 안 잡힘)

4장. 서버와 API: 요청 하나가 지나가는 길

4.1 서버와 클라이언트

식당에 비유하면 이렇습니다.

식당	다듬
손님	**클라이언트**: 웹 페이지, 크롬 확장, 맥 앱
주방	**서버**: `backend/` 의 파이썬 프로그램
주문서	**요청(request)**: "이 글을 검사해 줘"
음식	**응답(response)**: 지적 목록
메뉴판의 정해진 주문 방식	**API**

손님(화면)은 직접 요리하지 않습니다. 주문서만 써서 주방에 넘기고 음식을 받아 보여 줍니다. 그래서 **교정 로직은 서버 하나에만** 있고, 화면 세 개는 같은 서버를 부릅니다. 서버는 내 컴퓨터 안(`localhost:8000`)에서 돌기 때문에 글이 인터넷으로 나가지 않습니다.

4.2 API란

API는 "이렇게 주문하면 이렇게 답한다"는 약속입니다. 다듬의 API는 둘뿐입니다.

주소	방식	하는 일
`/api/check`	POST	글을 받아 지적 목록을 돌려줌
`/api/health`	GET	"살아 있나, 어느 층이 켜져 있나" 확인

POST는 "내용을 담아 보낸다", GET은 "그냥 달라고 한다"입니다.

4.3 주문서와 음식의 모양: `schemas.py`

```python
class CheckRequest(BaseModel):
    text: str = Field(max_length=20_000)

class Issue(BaseModel):
    id: int
    start: int
    end: int
    category: Category          # "spacing" | "spelling" | "josa" | "gramm
        ar" | "style"
    source: str
    original: str
    suggestions: list[str]
    message: str
    confidence: Confidence      # "high" | "medium" | "low"

class CheckResponse(BaseModel):
    issues: list[Issue]
    elapsed_ms: dict[str, float] = {}
```

- `BaseModel`(pydantic)은 "이 모양이 아니면 받지 않는다"는 검문소입니다. 모양이 틀린 주문서가 오면 서버가 안으로 들이지 않고 **422** 오류를 돌려줍니다.
- `max_length=20_000`: 글은 2만 자까지만. 원인은 서버 보호입니다. 2만 자는 서버 한 명을 약 4초 붙잡는 길이입니다.
- `Literal[...]`(글자 고정 목록): `category`는 다섯 값 중 하나여야 합니다.

실제로 있었던 사고: `style`이 빠져서 서버가 죽었다

`schemas.py` 맨 위 주석에 적힌 이야기입니다.

> 문체 일관성 검사기(`consistency.py`)는 처음부터 `category="style"`을 냈는데, 스키마의 `Category` 목록에 `style`이 없었습니다. 문체가 섞인 글을 검사하면 **`/api/check`가 500 오류로 죽었습니다**(2026-08-30 실측).

왜 테스트가 못 잡았나? 두 측정이 서로 다른 길을 탔기 때문입니다.

측정	길	결과
D 경로 게이트	검사기를 **직접** 불러서 잼	재현율 1.000 (정상으로 보임)
corpus-suite	**API를 거쳐서** 잼	예외로 죽어 0.000

두 숫자가 정반대인데 아무도 비교하지 않았습니다. 이후 “체커가 내는 모든 category가 스키마에 있는지 소스를 훑어 대조하는 계약 테스트”(`tests/test_category_contract.py`)가 생겼습니다. **교훈:** 같은 것을 두 군데서 다른 방법으로 재면, 둘이 어긋나는지 서로 대조해야 합니다.

4.4 서버 뼈대: `app/main.py`

```
app = FastAPI(title="다듬 API", version=APP_VERSION)
app.add_middleware(CORSMiddleware, allow_origins=..., allow_methods=["GE
        T","POST"], ...)
app.middleware("http")(rate_limit_middleware)
app.include_router(api_router, prefix="/api")
app.mount("/core", StaticFiles(directory=CORE_DIR), name="core")
app.mount("/", StaticFiles(directory=FRONTEND_DIR, html=True), name="front
        end")
```

- **FastAPI**: 파이썬으로 서버를 만드는 틀(프레임워크)입니다.
- **미들웨어**: 요청이 본 로직에 닿기 전에 지나가는 관문입니다. 여기서는 CORS와 레이트 리밋이 관문입니다.
- **라우터**: 주소별로 어느 함수가 맡을지 정한 배정표입니다. `prefix="/api"`라서 `check` 함수는 `/api/check`가 됩니다.
- **mount**: 폴더를 통째로 웹 주소에 연결합니다. 웹 앱의 `index.html`, `app.js`도 이 서버가 내줍니다.

등록 순서가 동작입니다. 주석의 경고: 정적 파일 mount는 “무엇이든 받는” 그물이라서 `/api` 라우터보다 먼저 등록하면 `/api/check` 요청까지 삼켜 버립니다. 그래서 라우터를 먼저 등록합니다.

CORS

브라우저는 보안상 “다른 곳에서 온 페이지가 이 서버를 부르는 것”을 기본으로 막습니다. 크롬 확장은 `chrome-extension://` 출처라서 막힙니다. 서버는 쿠키·로그인이 없으므로 모두 허용(`*`)이 안전하다고 판단했고, 운영에서는 `ALLOWED_ORIGINS` 환경변수로 좁힐 수 있습니다.

캐시 재검증 미들웨어

```
@app.middleware("http")
async def revalidate_static(request, call_next):
    response = await call_next(request)
    if not request.url.path.startswith("/api"):
        response.headers["Cache-Control"] = "no-cache"
    return response
```

정적 파일은 기본으로 캐시 지시가 없어서 브라우저가 낡은 `app.js`를 쓰는 일이 있었습니다(배포 후에도 옛 화면). `no-cache`는 "저장은 하되 쓰기 전에 서버에 확인"이라는 뜻이고, 안 바뀌었으면 가벼운 304 응답으로 끝납니다.

4.5 레이트리밋: `ratelimit.py`

한 사용자가 서버를 독점하지 못하게 합니다. 검사는 CPU를 쓰기 때문입니다.

```
IP마다: 60초 창 안에서 60번까지. 61번째부터 429 오류 + "몇 초 뒤 재시도" 안내.
```

작동 방식은 단순합니다.

```
_buckets[key] = (window_start, count)     # IP → (창 시작 시각, 횟수)
...
if now - window_start >= WINDOW:          # 창이 끝났으면 새로 시작
    window_start, count = now, 0
count += 1
if count > REQUESTS_PER_WINDOW:
    return JSONResponse(..., status_code=429, headers={"Retry-After":
        ...})
```

- `threading.Lock()`: 동시에 여러 요청이 `_buckets`를 고치면 값이 꼬이므로 한 번에 한 명만 들어가게 하는 자물쇠입니다.
- 5분마다 오래된 키를 청소합니다(메모리가 계속 늘지 않게).
- **한계가 코드에 적혀 있습니다.** 메모리 안에서만 세므로 서버가 여러 대면 공유가 안 됩니다. 한 대로 쓰는 지금은 충분하다는 판단입니다.

어느 IP인가: `clientip.py`

프록시 뒤에서는 모든 요청의 접속자가 프록시로 보입니다. 원래 IP는 `X-Forwarded-For` 헤더에 있는데, **이 헤더는 누구나 위조할 수 있습니다.** 위조를 믿으면 요청마다 헤더를 바꿔 한도를 무한히 우회합니다.

설정	동작
기본	헤더를 믿지 않고 직접 접속한 IP만 사용
`TRUSTED_PROXIES=IP목록`	접속자가 그 목록일 때만 헤더를 믿음 (권장)
`TRUST_PROXY=1`	옛 방식(누구 헤더든 믿음), 경고 출력

예전엔 기본값이 "믿음"이라 우회가 가능했고, 기본을 "안 믿음"으로 뒤집었습니다. **안전한 쪽이 기본값**이어야 한다는 원칙입니다.

4.6 `/api/check` 한 줄씩

```
@router.post("/check", response_model=CheckResponse)
def check(request: CheckRequest, x_dadeum_client: str | None = Header(defa
        ult=None)):
    _note_client(x_dadeum_client)
    text = request.text
    if not text.strip():
        return CheckResponse(issues=[])

    result = analyze_long(text)
    return CheckResponse(
        issues=[build_issue(idx, r) for idx, r in enumerate(result.issue
        s)],
        elapsed_ms=result.elapsed_ms,
    )
```

1. `@router.post("/check")` : 데코레이터. "아래 함수를 `/check` POST 주문 담당으로 등록하라."
2. `request: CheckRequest` : 검문소를 통과한 주문서가 들어옵니다.
3. `x_dadeum_client` : 헤더 `X-Dadeum-Client`를 읽습니다. 확장이 "나 여기 있다"고 알리는 용도입니다(4.8).
4. 빈 글이면 바로 빈 목록.
5. `analyze_long(text)`가 엔진을 돌립니다(5장).

6. `build_issue(idx, r)`로 내부 자료(`RawIssue`)를 응답 모양(`Issue`)으로 바꾸고, 목록 번호를 `id`로 붙입니다.

실제 응답은 이런 JSON입니다(예문 `학교을 갔다`, 제가 `check()`를 직접 불러 얻은 값).

```
{"issues": [{"id": 0, "start": 2, "end": 3, "category": "josa",
             "source": "josa_rule", "original": "을", "suggestions": ["를"],
             "message": "받침 여부를 확인하면 '를'이 맞는 조사입니다.",
             "confidence": "high"}],
 "elapsed_ms": {"layer1": 4278.2, "layer2": 2.0}}
```

- `message`의 끝 `이/가`는 **제안한 조사 자신의 받침**으로 고릅니다(`_issue()`의 `marker`). `를`은 받침 ㄹ이 있어 `'를'이`, `가`는 받침이 없어 `'가'가`가 됩니다. 설명문조차 같은 받침 계산으로 맞춰 쓰는 것입니다.
- `layer1`이 4278ms로 큰 것은 **서버를 막 띄운 뒤 첫 요청**이라서입니다(사전·모델을 처음 읽는 비용). 두 번째부터는 한 자릿수 ms입니다. 이 값은 `/check` 응답에 그대로 실려서 "어느 층이 시간을 쓰는지" 보는 용도입니다.

4.7 긴 글: `analyze_long`

```
CHUNK_LIMIT = 4000

def analyze_long(text, use_l2=True):
    if len(text) <= CHUNK_LIMIT:
        return analyze(text, use_l2)
    ...
    for start, end in _chunk_bounds(text, CHUNK_LIMIT):
        part = analyze(text[start:end], use_l2)
        for r in part.issues:
            issues.append(replace(r, start=r.start + start, end=r.end + st
        art))
```

4000자를 넘으면 **문장 경계에서** 잘라 조각마다 따로 돌리고, 위치(start, end)를 원문 기준으로 되돌립니다(조각의 시작 위치를 더함).

왜 필요했나? 예전에는 스키마 상한이 5,000자였고, 클라이언트들이 글을 자르지 않고 통째로 보냈습니다. **긴 문서는 422로 튕겨서 검사가 아예 안 됐습니다**(데스크톱·확장·웹 모두 해당). 이후 엔진이 직접 나눠 돌리게 바꿨고 상한을 2만 자로 올렸습니다.

`_chunk_bounds`의 핵심:

```
for sent in _kiwi.split_into_sents(text):
    if sent.end - start > limit and cur > start:
        bounds.append((start, cur))     # 여기까지 한 조각
        start = cur
    while sent.end - start > limit:     # 마침표 없이 이어진 아주 긴 덩어리
        bounds.append((start, start + limit))
        start += limit
    cur = sent.end
```

- Kiwi의 `split_into_sents`가 문장 경계를 줍니다.
- 문장이 상한을 넘기면 그 문장만 억지로 4000자씩 자릅니다.

한계(주석에 있음): 문체 일관성은 글 전체를 봐야 하는데 조각 안에서만 판정합니다.

4.8 `/api/health`: 지금 어느 층이 켜져 있나

```
{"status": "ok", "version": ..., "model_family": "gyeol-1",
 "layer2_pairs": ["an", "doe", "den", "roseo"],
 "boundary_model": True, "lexicon": 1806050,
 "spacing_tagger": False, "extension_live": False}
```

왜 이렇게 자세한가: 이 엔진의 층은 전부 "선택"입니다. 모델 파일이 없거나 torch가 없으면 그 층만 조용히 빠집니다. 그러면 "내 컴퓨터에서는 잡히는데 배포본에서는 안 잡힌다" 같은 일이 생기고, 원인을 찾기 어렵습니다. `/api/health`가 각 층의 적재 여부를 답해 주므로 **결과가 다를 때 가장 먼저 볼 곳**입니다.

`extension_live`는 맥 앱이 읽습니다. 맥 앱은 크롬에서 확장이 담당하므로 "확장이 살아 있을 때만" 비켜섭니다. 확장이 `check`/`health` 요청마다 `X-Dadeum-Client: extension` 헤더를 달고, 서버가 마지막으로 본 시각을 기억해 **180초 안이면 살아 있다**고 답합니다. (설치 안 된 확장을 믿고 맥 앱이 비켜서면 크롬에서 아무 교정도 안 나오는 일이 있었습니다. 그것을 막는 장치입니다.)

4.9 직접 따라 하기

서버를 띄우고 직접 주문해 볼 수 있습니다.

```
cd dadeum/backend
source .venv/bin/activate
uvicorn app.main:app --port 8000

# 다른 터미널에서
curl -s localhost:8000/api/health
curl -s -X POST localhost:8000/api/check \
     -H "Content-Type: application/json" \
     -d '{"text": "학교을 갔다"}'
```

4.10 확인 문제

1. 글이 25,000자이면 어떤 응답이 올까? (422: 스키마 상한 2만 자 초과)
2. 같은 IP가 1분에 61번 요청하면? (61번째부터 429 + Retry-After)
3. 서버가 `style`로 죽었던 이유는? (스키마의 Category 목록에 style이 없었음)

5장. 엔진 뼈대: 후보를 모으고 정리하는 구조

6~10장에서 개별 검사기를 하나씩 봅니다. 그 전에 **검사기들을 감싸는 틀**을 이해해야 합니다. 이 틀이 `backend/app/engine.py`(183줄)와 `backend/app/checkers/base.py`, `sources.py`입니다.

5.1 한 문장으로

> **검사기 21개가 각자 "여기 틀린 것 같다"는 후보를 내면, 겹치는 후보를 우선순위로 정리해서 한 목록으로 돌려준다.**

> 문장 → ① 분석 → ② 후보 → ③ 병합 → 지적 목록

5.2 왜 "하나의 흐름"인가: 만들어진 이유

`engine.py` 맨 위 설명문에 이유가 적혀 있습니다.

> 이 흐름이 예전엔 두 군데에 따로 적혀 있었다: `app/api.py`의 `/check`와 `eval/qa/common.py`의 `run_local`. 한쪽에만 검사기를 더한 채로 어긋나, **문체 일관성 재현율이 평가에서 0으로 찍힌 적이 있다**(측정 도구가 제품과 다른 코드를 밟았다).

즉 "제품이 돌리는 길"과 "성적을 재는 길"이 서로 다른 복사본이었습니다. 제품에는 검사기를 넣었는데 평가 쪽에는 안 넣어서, 평가는 "그 검사기는 아무것도 못 잡는다"고 보고했습니다. 지금은 둘 다 `engine.analyze()` 한 함수만 부릅니다. **검사기를 더하거나 빼는 곳이 `GENERATORS` 목록 한 줄**입니다.

> 이 원칙의 뿌리는 4장에서 본 `style` 사건과 같습니다. "재는 길과 쓰는 길이 같아야 한다."

5.3 자료: `RawIssue`

모든 검사기가 같은 모양으로 후보를 냅니다(`checkers/base.py`).

필드	뜻	예
`start`, `end`	원문 위치 `[start, end)`	`2`, `3`
`category`	종류	`josa`
`source`	어느 규칙이 냈나	`josa_rule`
`original`	원문의 그 부분	`을`
`suggestions`	고칠 말 목록 (비어 있을 수 있음)	`["를"]`
`message`	사람에게 보여 줄 설명	
`confidence`	`high` / `medium` / `low`	`high`

`suggestions`가 비어 있다는 것은 "여기가 수상하다"만 말하고 고칠 말은 없다는 뜻입니다. 이상탐지, 문체 일관성이 그렇습니다.

5.4 검사기 등록: `GENERATORS`

```python
@dataclass(frozen=True)
class Generator:
    name: str
    layer: int      # 1 = 규칙·로컬 모델, 2 = 혼동쌍 분류기
    run: Callable[[str, object], list[RawIssue]]

GENERATORS = [
    Generator("spacing", 1, check_spacing),
    Generator("dictionary", 1, lambda t, k: check_dictionary(t)),
    Generator("morph", 1, check_morph_rules),
    Generator("josa", 1, check_josa),
    Generator("jamo", 1, check_jamo),
    ...
    Generator("confusion", 2, lambda t, k: check_confusion(t)),
    Generator("boundary", 2, check_boundary),
]
```

모든 검사기는 **같은 약속**을 지킵니다: `(글, Kiwi) → 후보 목록`. 어떤 검사기는 Kiwi가 필요 없어서 `lambda t, k: check_dictionary(t)`처럼 `k`를 버립니다.

`layer`는 소요 시간을 층별로 나눠 재기 위한 꼬리표입니다(응답의 `elapsed_ms`). 이름이 "2층"이지만 실행 순서가 아니라 **분류**입니다. 목록 순서대로 하나씩 실행됩니다.

21개의 이름과 담당

이름	담당	장
spacing	띄어쓰기 (태거 + 삭제 + 규칙 30여 개)	10
dictionary	사전 규칙 474개 + `에요/예요`	6
morph	형태소 고쳐 활용형 되살리기	6
josa	받침에 따른 조사	6
jamo	흩어진 자모 합치기	6
dependent_noun	의존명사 띄어쓰기	6
particle_spacing	조사를 앞말에 붙이기	6
yul_ryul	율/률	6
quotation	간접 인용 `-라고` 겹침	6
question_mark	의문문 부호	6
honorific	사물 존대	6
animacy	에게/에	6
negative_polarity	결코·전혀 + 부정	6
tense	시제 호응	6
causative	불필요한 `-시키다`	6
subject_predicate	주술 호응	6
consistency	문체 섞임	6
typo	키보드 오타	9
anomaly	낯선 어절	9
confusion	되/돼 등 혼동쌍 모델	7
boundary	단어 경계 모델	8

5.5 `analyze()` 한 줄씩

```
def analyze(text, use_l2=True):
    shared = _SharedAnalysis(_kiwi)
    candidates = []
    elapsed = {1: 0.0, 2: 0.0}
    for gen in GENERATORS:
        if gen.layer == 2 and not use_l2:
            continue
        t0 = perf_counter()
        candidates += gen.run(text, shared)
        elapsed[gen.layer] += perf_counter() - t0
    return Analysis(
        issues=merge_and_dedupe(candidates),
        elapsed_ms={"layer1": ..., "layer2": ...},
    )
```

1. `_SharedAnalysis(_kiwi)` : 이 요청 동안만 쓰는 분석 공유 캐시(3장).
2. `for gen in GENERATORS` : 검사기를 **등록된 순서대로** 하나씩.
3. `candidates += gen.run(text, shared)` : 후보를 한 바구니에 쏟아 넣습니다. 이 시점에는 정리하지 않습니다.
4. `perf_counter()` : 시계. 실행 전후를 재서 층별 시간을 모읍니다.
5. 마지막에 `merge_and_dedupe` 로 한 번에 정리합니다.

`use_l2=False` 로 2층을 끄고 1층만 돌릴 수 있습니다(평가에서 층별 기여를 따로 재기 위해).

> **순서는 동작의 일부입니다.** 병합은 "같은 자리·같은 길이면 먼저 온 쪽이 남는" 안정 정렬이라서, 목록 순서를 바꾸면 출력이 바뀝니다. 그래서 목록 순서도 코드의 일부로 취급합니다.

5.6 병합: `merge_and_dedupe` (엔진의 심장)

검사기들은 서로를 모릅니다. 그래서 같은 자리를 둘 이상이 지적하는 일이 흔합니다. 예문 `안되요` 에서는 사전 규칙이 `되요→돼요` 를, 혼동쌍 모델이 `되→돼` 를 같은 자리에 냈습니다. 둘 다 보여 주면 겹친 밑줄이 되므로 **하나만** 남깁니다.

알고리즘

```
ordered = sorted(issues, key=lambda i: (i.start, -(i.end - i.start)))
kept = []
cursor = -1
for issue in ordered:
    if issue.start >= cursor:           # 앞의 것과 안 겹침
        kept.append(issue); cursor = issue.end
        continue
    prev = kept[-1]                     # 겹침 → 승부
    if _has_fix(issue) != _has_fix(prev):
        if _has_fix(issue):
            kept[-1] = issue; cursor = issue.end
        continue                        # (제안 있는 쪽이 이김)
    if SOURCE_PRIORITY[issue.source] > SOURCE_PRIORITY[prev.source]:
        kept[-1] = issue; cursor = issue.end
return _drop_noise_adjacent_to_fixed_typos(kept)
```

규칙을 말로 풀면

1. **정렬**: 시작 위치가 빠른 순, 같으면 긴 것 먼저.
2. 하나씩 보면서, 직전에 남긴 것과 **안 겹치면 남깁니다.**
3. **겹치면 승부**:
    - **(가)** 한쪽만 고칠 말이 있으면 → **고칠 말이 있는 쪽**이 이긴다.
    - **(나)** 둘 다 있거나 둘 다 없으면 → **출처 우선순위 숫자가 큰 쪽**.
    - **(다)** 우선순위도 같으면 → **먼저 온 쪽**이 남는다(바꾸지 않으므로).

여섯 가지 상황을 직접 돌려 본 결과

#	입력	결과	이유
1	`(0–2 dict)` `(5–7 josa)`	둘 다 남음	안 겹침
2	`(26–28 dict, 제안)` `(26–27 ml_confusion, 제안)`	**dict**	둘 다 제안, 우선순위 5 > 4
3	`(30–32 anomaly, 제안 없음)` `(30–32 typo, 제안 있음)`	**typo**	(가) 제안 있는 쪽
4	`(0–2 kiwi_space)` `(0–2 tagger_space)`	**kiwi_space**	우선순위 0 = 0, 먼저 온 쪽
5	`(2–3 josa)` `(0–5 dict)`	**(0-5 dict)**	긴 쪽이 먼저 정렬됨, josa(2)는 dict(5)에 짐
6	`(0–3 dict 희한, high)` `(3–4 kiwi_space, medium)`	**dict만**	아래 5.8

왜 “제안 있는 쪽이 먼저”인가 — 실제 사고

코드 주석(2026-08-30)입니다.

> 교정을 만들지 않는 층(이상탐지, 문체 일관성)은 “여기가 수상하다”만 말한다. 그런 지적이 어절 전체를 덮으면 그 안의 진짜 교정을 밀어낼 수 있다. 실제로 그랬다: `자률신경` **을 이상탐지가 통째로 잡아** `률→율` **교정이 사라졌고** 회귀 레지스트리가 그걸 잡아냈다. 우선순위 숫자만으로는 못 막는다. **조언은 교정에 양보한다.**

이상탐지의 우선순위가 0이고 `yul_ryul_rule`도 0이라 숫자만으로는 비길 때가 있었던 것입니다. 그래서 “숫자 비교 이전에 제안 유무로 먼저 가른다”는 규칙이 앞에 들어갔습니다.

5.7 출처 우선순위표: `sources.py`

58개 출처가 한 표에 적혀 있습니다. 구조는 이렇습니다.

```python
@dataclass(frozen=True)
class Source:
    name: str       # "josa_rule"
    layer: str      # "rule" | "model" | "advice"
    priority: int
    basis: str      # 근거 설명
```

층(layer)의 뜻

층	뜻
`rule`	결정적 규칙. 같은 입력이면 늘 같은 출력
`model`	통계·학습 모델 (혼동쌍, 태거, 경계 판정기 등)
`advice`	교정을 만들지 않고 "확인해 보라"만 말함

우선순위

값	출처	이유
6	jamo, quotation, honorific, animacy, negative_polarity, tense, causative, agreement	문법 규칙이 문맥을 보고 확정한 것. 가장 특수하고 신뢰 높음
5	dict_rule	국립국어원 기준 사전 규칙
4	ml_confusion	혼동쌍 학습 모델 (형태론 확정 포함)
3	dependent_noun_rule, particle_spacing_rule	의존명사·조사 붙이기
2	josa_rule	받침 조사
1	typo_rule, boundary_model	오타·경계 모델
0	나머지 (띄어쓰기 세부 규칙 대부분, kiwi_space, tagger_space, consistency, anomaly)	기본값

원칙: **"더 특수하고 더 선별된 근거가 일반적인 것을 이긴다."**

표 없이 쓰던 시절의 사고

`sources.py` 맨 위 주석입니다.

> 예전에는 우선순위가 `base.py` 의 딕셔너리에만 있었고, 2026-09-13에 추가한 띄어쓰기 규칙 11종은 **그 표에 없는 채로 기본값 0으로 돌았다.** 출처를 더하는 곳을 한 군데로 모으고, **표에 없는 출처가 나오면 테스트가 실패하게 했다** (`tests/test_engine.py`).

"규칙을 더했는데 우선순위를 안 적어서 조용히 0이 된" 사고입니다. 기본값에 의존하면 아무도 모르게 틀립니다. 그래서 **"모르는 출처가 나오면 시끄럽게 실패"**로 바꿨습니다.

```
def priority(name):
    s = SOURCES.get(name)
    return s.priority if s else 0
```

(이 함수 자체는 여전히 기본값 0을 주지만, 테스트가 "엔진이 내는 모든 source가 표에 있다"를 검증합니다.)

5.8 마지막 청소: `_drop_noise_adjacent_to_fixed_typos`

```
anchors = [i for i in issues if i.source == "dict_rule" and i.confidence =
        = "high" and i.suggestions]
return [issue for issue in issues
        if not (issue.source in ("kiwi_space", "tagger_space", "kiwi_typ
        o")
                and issue.confidence != "high"
                and any(issue.start == a.end or issue.end == a.start for a
        in anchors))]
```

상황: `희안하게` 는 맞춤법 오류(`희한하게`)입니다. 사전 규칙이 `희안→희한` 을 high로 잡습니다. 그런데 틀린 철자 때문에 Kiwi가 이 단어를 엉뚱하게 해석해서 바로 옆에 "`하` 를 따로 띄우자"는 쓸데없는 제안이 같이 나옵니다.

해결: 확실한 사전 규칙 교정의 **바로 옆에 붙어 있는** 애매한 띄어쓰기·오타 후보는 버립니다. 맞춤법을 고치고 나면 그 전제(틀린 철자)가 사라지므로 쓸데없는 제안이 되기 때문입니다. `i.start == a.end or i.end == a.start` 는 "앞뒤로 딱 붙어 있다"를 뜻합니다.

5.9 예문 전체 추적

```
학교을 가는길에 친구를 만낫는데 그렇게 하면 안되요. 매읽 그 영화 봤든 사람이 너무많아서 놀랐다.
```

② 후보 (검사기가 낸 것, 실제 실행)

```
[spacing    ] 47–48 '많'  → ' 많'   high  degree_adverb_rule
[spacing    ] 25–26 '안'  → '안 '   high  negation_context_rule
[spacing    ]  6–7  '길'  → ' 길'   low   tagger_space
[dictionary ] 26–28 '되요' → '돼요'  high  dict_rule
[josa       ]  2–3  '을'  → '를'    high  josa_rule
[typo       ] 30–32 '매읽' → '매일'  low   typo_rule
[anomaly    ] 13–17 '만낫는데' → []  low   anomaly
[anomaly    ] 30–32 '매읽'  → []    low   anomaly
[confusion  ] 26–27 '되'  → '돼'    high  ml_confusion
[confusion  ] 39–40 '든'  → '던'    high  ml_confusion
```

후보 10개 (이 환경은 띄어쓰기 태거가 켜져 있습니다. 꺼진 환경에서는 `길` 항목이 없습니다).

③ 병합 뒤 (8개)

```
 2–3   '을'  → '를'    high  josa
 6–7   '길'  → ' 길'   low   spacing  (tagger)
13–17  '만낫는데' → [] low   spelling (anomaly)
25–26  '안'  → '안 '   high  spacing
26–28  '되요' → '돼요'  high  spelling (dict_rule)
30–32  '매읽' → '매일'  low   spelling (typo_rule)
39–40  '든'  → '던'    high  grammar (ml_confusion)
47–48  '많'  → ' 많'   high  spacing
```

빠진 두 개: - `(26–27 되→돼, ml_confusion)` : `(26–28 되요→돼요, dict_rule)`에 졌습니다. 정렬에서 같은 start(26)이면 긴 쪽(26-28)이 먼저 오고, 짧은 쪽은 겹치므로 우선순위 5 > 4로 탈락. - `(30–32 매읽, anomaly)` : `typo_rule`이 고칠 말을 가졌으므로 탈락(가).

한 가지 눈여겨볼 점: 서로 다른 길이 같은 곳을 가리킨다

`안되요`는 두 가지가 동시에 맞습니다: `안` 뒤에 공백(`안 되요`)과 `되요→돼요`. 둘은 위치가 달라서(25-26, 26-28) 겹치지 않고 **둘 다 남습니다.** "모두 고치기"가 앞에서부터 하나씩 적용하면 `안 돼요`가 됩니다(12장).

5.10 확인 문제

1. 겹치는 두 후보가 모두 제안이 있고 우선순위가 같으면? (먼저 정렬된 것이 남음, 정렬 키는 시작 위치·긴 것 먼저)
2. 이상탐지(제안 없음, 우선순위 0)가 사전 규칙(제안 있음)과 겹치면 누가 이기나? (사전 규칙. 우선순위 비교 이전에 “제안 있음”이 이긴다)
3. `GENERATORS`의 순서를 바꾸면 안 되는 이유? (같은 자리·같은 길이·같은 우선순위일 때 먼저 온 쪽이 남기 때문)